Python for AI Developers
A Guide for Angular Developers

Basics | OOP | Async | Pydantic | FastAPI | NumPy | LLM APIs | LangChain | Agents | RAG

You already know programming very well from 7 years of Angular and TypeScript. This guide teaches the **Python you need for AI development** in **73 short questions**, starting from zero and building up to calling LLMs, LangChain, agents and RAG.

Every question has the same parts:

Definition	What it is, in one or two simple sentences.
Explanation	Why it matters and how it is used, in plain language.
Python	A small, runnable example.
TypeScript	The same idea in TypeScript/Angular, so you can connect it to what you already know.
Tip	A common mistake or a useful extra.

Best way to learn: type every Python example yourself (in a notebook or the python shell) instead of only reading it. At the end you'll find a TypeScript-to-Python cheat sheet and a learning roadmap.

Contents

1. Getting Started . . . 5

Q1. Why is Python the main language for AI development? . . . 5
Q2. How do I run Python code? . . . 5
Q3. What is a virtual environment (venv)? . . . 5
Q4. What are pip and requirements.txt? . . . 6
Q5. What is a Jupyter Notebook? . . . 6
Q6. Indentation instead of { } - how does it work? . . . 7

2. Variables & Data Types . . . 8

Q7. How do I create variables? Is there let/const? . . . 8
Q8. What are the basic data types? . . . 8
Q9. What is None, and how do I check it? . . . 8
Q10. What are f-strings? . . . 9
Q11. Which string methods will I use most? . . . 9
Q12. What are truthy and falsy values? . . . 9
Q13. What are the logical and comparison operators? . . . 10

3. Collections: list, tuple, dict, set . . . 11

Q14. What is a list? . . . 11
Q15. What is slicing? . . . 11
Q16. What is a dict (dictionary)? . . . 12
Q17. What is a tuple? . . . 12
Q18. What is a set? . . . 13
Q19. What is unpacking (destructuring) and the * / ** spread? . . . 13
Q20. What are list and dict comprehensions? . . . 13

4. Control Flow . . . 14

Q21. How do if / elif / else and the ternary work? . . . 14
Q22. How do for loops work? What are range, enumerate and zip? . . . 14
Q23. while, break and continue? . . . 15
Q24. What is match / case? . . . 15

5. Functions . . . 16

Q25. How do I define a function? . . . 16
Q26. What are *args and **kwargs? . . . 16
Q27. What are lambda functions? . . . 17
Q28. What are type hints? . . . 17
Q29. What is a docstring? . . . 17
Q30. What is variable scope? What are global and nonlocal? . . . 18
Q31. What is a closure? . . . 18

6. Classes & Objects (OOP) . . . 19

Q32. How do I write a class? . . . 19
Q33. Is there public / private? . . . 19
Q34. How does inheritance work? . . . 20
Q35. What are 'dunder' (magic) methods like __str__ and __len__? . . . 20
Q36. What are @property, @staticmethod and @classmethod? . . . 21
Q37. What is a dataclass? . . . 21
Q38. Abstract classes and Protocols - how do I write an 'interface'? . . . 22

7. Modules, Packages & Imports . . . 23

Q39. How do imports work? . . . 23
Q40. What is a package and __init__.py? . . . 23
Q41. What does if __name__ == "__main__": mean? . . . 23
Q42. Which standard library modules should I know? . . . 24

8. Error Handling . . . 25

Q43. How do try / except / finally work? . . . 25
Q44. How do I raise my own errors? . . . 25
Q45. What is 'with' (a context manager)? . . . 26

9. Files, JSON & Environment Variables . . . 27

Q46. How do I read and write text files? . . . 27
Q47. How do I work with JSON? . . . 27
Q48. What is pathlib? . . . 28
Q49. How do I store API keys safely (environment variables, .env)? . . . 28

10. Must-Know Advanced Python . . . 29

Q50. What is an iterator / iterable? . . . 29
Q51. What is a generator and yield? . . . 29
Q52. What is a decorator? . . . 30
Q53. What is @lru_cache? . . . 30
Q54. What is async / await in Python? . . . 31
Q55. Threads, processes or async - what is the GIL? . . . 31
Q56. How do I log properly instead of using print? . . . 32

11. Key Libraries for AI Developers . . . 33

Q57. What is Pydantic and why is it everywhere in AI code? . . . 33
Q58. How do I call a REST API (requests / httpx)? . . . 33
Q59. What is FastAPI? . . . 34
Q60. What is NumPy? . . . 34
Q61. What is pandas? . . . 35

12. Python for LLM & AI Apps . . . 36

Q62. How do I call an LLM from Python (OpenAI / Azure OpenAI SDK)? . . . 36
Q63. How do I stream the answer word by word? . . . 36

Q64. How do I create embeddings and compare them? . . . 37
Q65. What does a basic LangChain chain look like? . . . 37
Q66. How do I get JSON / typed objects back from an LLM (structured output)? . . . 38
Q67. What is a tool, and how does an agent use it? . . . 38
Q68. What does a minimal RAG flow look like in Python? . . . 39
Q69. How do I handle errors and retries when calling AI APIs? . . . 39

13. Testing & Code Quality . . . 40

Q70. How do I write tests with pytest? . . . 40
Q71. How do I test code that calls an LLM? . . . 40
Q72. Which code quality tools should I use? . . . 41
Q73. What does a typical AI project folder structure look like? . . . 41

14. TypeScript to Python Cheat Sheet . . . 42

15. Your 6-Week Learning Roadmap . . . 44

1. Getting Started

Why Python, and how to set up a project. Think of this as 'npm install + ng serve' for Python.

Q1. Why is Python the main language for AI development?

DEFINITION

Python is a simple, readable language with the biggest collection of AI and data libraries.

Almost every AI tool is built for Python first: OpenAI and Azure OpenAI SDKs, LangChain, LangGraph, NumPy, pandas, PyTorch, Hugging Face. As an AI integration developer you mostly **connect** these libraries to data, APIs and UIs - and your Angular experience covers the UI side.

Tip: Good news: an AI integration job uses maybe 30% of Python. You don't need to master all of it.

Q2. How do I run Python code?

DEFINITION

Python is an interpreted language: you run a .py file directly, no compile/build step.

Save code in a file and run it with `python file.py`. You can also type `python` to open an interactive shell (REPL) and try lines one by one - great for learning.

PYTHON EXAMPLE

```
# hello.py
print("Hello, AI world!")

# terminal
python hello.py
python -m backend.main      # run a module inside a package
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
// TypeScript needs a compile step
tsc hello.ts && node hello.js
```

Q3. What is a virtual environment (venv)?

DEFINITION

A private folder of installed packages for ONE project, so projects don't break each other.

It is the Python version of a project's `node_modules` folder. Without it, every package is installed globally. Create one per project and **activate** it before installing.

PYTHON EXAMPLE

```
python -m venv .venv            # create (once)
.venv\Scripts\activate         # activate on Windows
source .venv/bin/activate       # activate on Mac/Linux
deactivate                      # leave it
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
// Node: node_modules is per project automatically
npm install
```

Tip: Add `.venv/` to .gitignore, just like node_modules.

Q4. What are pip and requirements.txt?

DEFINITION

pip = Python's package installer (like npm). requirements.txt = the list of packages (like package.json dependencies).

PYTHON EXAMPLE

```
pip install langchain fastapi       # install packages
pip install -r requirements.txt     # install everything in the list
pip freeze > requirements.txt       # save installed versions

# requirements.txt
fastapi>=0.115
langchain>=1.0
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
npm install @angular/core rxjs
npm install            // from package.json
```

Tip: Modern tools: `uv` (very fast, like pnpm) and Poetry (with a lock file, like package-lock.json).

Q5. What is a Jupyter Notebook?

DEFINITION

An interactive document where you write and run code in small cells and see the output right below.

Data scientists and AI developers use notebooks to experiment: try a prompt, look at embeddings, test a retriever. VS Code opens `.ipynb` files directly. For real applications you still write `.py` files.

PYTHON EXAMPLE

```
# Cell 1
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o-mini")

# Cell 2 (run again and again while you tweak the prompt)
llm.invoke("Explain RAG in one sentence").content
```

Tip: Think of it like a StackBlitz playground for Python.

Q6. Indentation instead of { } - how does it work?

DEFINITION

Python uses indentation (4 spaces) to mark code blocks. There are no curly braces and no semicolons.

A colon : starts a block; every line in the block is indented. Wrong indentation is a real error, not just bad style.

PYTHON EXAMPLE

```
def greet(name):
    if name:
        print("Hello", name)
    else:
        print("Hello stranger")
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
function greet(name: string) {
  if (name) {
    console.log('Hello', name);
  } else {
    console.log('Hello stranger');
  }
}
```

Tip: Comments start with #. Multi-line notes use triple quotes """ ... """.

2. Variables & Data Types

The building blocks. Most of it will feel familiar from TypeScript.

Q7. How do I create variables? Is there let/const?

DEFINITION

Just write `name = value`. No let, const or var. The type is decided by the value (dynamic typing).

Python has no `const`. By convention, names in ALL_CAPS mean 'don't change this' (like `TOP_K = 5` in a config file). Variable names use **snake_case**, not camelCase.

PYTHON EXAMPLE

```
user_name = "Rushabh"     # str
age = 30                  # int
temperature = 0.2         # float
is_active = True          # bool (capital T)
result = None             # 'no value' (like null)
TOP_K = 5                 # constant by convention
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
let userName = 'Rushabh';
const age = 30;
let result: string | null = null;
```

Q8. What are the basic data types?

DEFINITION

str (text), int (whole number), float (decimal), bool (True/False), None (no value).

Unlike TypeScript's single `number`, Python separates int and float. Use `type(x)` to check a type and `int()`, `float()`, `str()` to convert.

PYTHON EXAMPLE

```
type(5)          # <class 'int'>
type(5.0)        # <class 'float'>
int("42")        # 42   (like Number('42') / parseInt)
str(42)          # "42"
float("0.75")    # 0.75
10 / 3           # 3.333... (always float)
10 // 3          # 3        (integer division)
2 ** 3           # 8        (power)
```

Q9. What is None, and how do I check it?

DEFINITION

`None` is Python's single 'nothing' value - the same idea as `null` and `undefined` together.

Check it with `is None` / `is not None`, not with `==`.

PYTHON EXAMPLE

```
answer = None
if answer is None:
    print("No answer yet")

year = filters.year if filters.year is not None else 2025
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
if (answer === null || answer === undefined) { ... }
const year = filters.year ?? 2025;
```

Q10. What are f-strings?

DEFINITION

Strings that start with `f` and can include variables inside `{ }`. Same as TS template literals.

You will use them constantly to build prompts.

PYTHON EXAMPLE

```
name = "Priya"
days = 24
print(f"Hi {name}, you have {days} leave days")
print(f"Score: {0.87654:.2f}")       # 2 decimals -> Score: 0.88

prompt = f"""Answer using this context:
{context}
Question: {question}"""           # multi-line f-string
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
console.log(`Hi ${name}, you have ${days} leave days`);
```

Q11. Which string methods will I use most?

DEFINITION

Built-in functions on text: clean, split, join, search and replace.

PYTHON EXAMPLE

```
text = "  Hello World  "
text.strip()                 # "Hello World"   (trim)
text.lower()                 # "  hello world  "
"a,b,c".split(",")          # ['a', 'b', 'c']
", ".join(["a", "b"])        # "a, b"  (note: join is on the separator!)
"VPN" in "Use VPN at home"   # True  (like includes)
text.replace("World", "AI")
text.startswith("  He")       # True
len(text)                    # 15   (like .length)
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
text.trim(); text.toLowerCase(); 'a,b,c'.split(',');
['a','b'].join(', '); 'Use VPN'.includes('VPN');
```

Q12. What are truthy and falsy values?

DEFINITION

Values that count as False in an `if`: False, None, 0, 0.0, "" (empty string), [] , {} , set().

Very similar to JavaScript, with one big difference: **empty lists and dicts are falsy** in Python (in JS `[]` is truthy!).

PYTHON EXAMPLE

```
docs = []
if not docs:            # True because the list is empty
    print("Nothing found")

name = user_input or "Guest"     # default when empty (like ||)
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
if (docs.length === 0) { ... }   // [] is truthy in JS!
```

Q13. What are the logical and comparison operators?

DEFINITION

Python uses words: `and`, `or`, `not`. Equality is `==` (there is no `===`).

`==` compares values, `is` checks if two names point to the very same object (used for None). Comparisons can be chained.

PYTHON EXAMPLE

```
if age >= 18 and is_active:
    ...
if not allowed or blocked:
    ...
if 0 < score <= 1:          # chained comparison
    ...
"hr" in ["hr", "it"]        # True  (membership)
"x" not in "hello"          # True
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
if (age >= 18 && isActive) { ... }
if (!allowed || blocked) { ... }
```

3. Collections: list, tuple, dict, set

The four containers you use every day. In AI apps everything is lists of messages, dicts of metadata and lists of vectors.

Q14. What is a list?

DEFINITION

An ordered, changeable collection - Python's version of an array.

PYTHON EXAMPLE

```
chunks = ["chunk A", "chunk B"]
chunks.append("chunk C")       # push
chunks.extend(["D", "E"])      # push many
first = chunks[0]               # "chunk A"
last = chunks[-1]               # last item (negative index!)
chunks.pop()                    # remove + return last
len(chunks)                     # length
sorted(chunks)                  # new sorted list
"chunk A" in chunks             # True
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
const chunks = ['chunk A', 'chunk B'];
chunks.push('chunk C'); chunks.at(-1); chunks.length;
```

Q15. What is slicing?

DEFINITION

Taking a part of a list or string with `[start:end]` (end is not included).

Very common in AI code, e.g. 'only keep the last 6 messages of history'.

PYTHON EXAMPLE

```
items = [10, 20, 30, 40, 50]
items[1:3]      # [20, 30]
items[:2]       # [10, 20]       first 2
items[-2:]      # [40, 50]       last 2
items[::-1]     # [50, 40, 30, 20, 10]  reversed

recent_history = history[-6:]   # last 6 messages
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
items.slice(1, 3); items.slice(-2);
```

Q16. What is a dict (dictionary)?

DEFINITION

A collection of key -> value pairs. Like a JavaScript object or Map.

JSON from APIs becomes dicts in Python. Access keys with `d["key"]` (not `d.key`). Use `.get()` when the key might be missing - it returns None (or your default) instead of crashing.

PYTHON EXAMPLE

```
metadata = {"fileName": "leave.pdf", "page": 3}
metadata["page"]                    # 3
metadata["year"]                    # KeyError! key missing
metadata.get("year")                # None
metadata.get("year", 2025)          # 2025 (default)
metadata["department"] = "hr"       # add / update
"page" in metadata                  # True (key exists?)
for key, value in metadata.items():  # like Object.entries
    print(key, value)
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
const metadata = { fileName: 'leave.pdf', page: 3 };
metadata.year ?? 2025;
Object.entries(metadata).forEach(([k, v]) => ...);
```

Tip: `.keys()`, `.values()`, `.items()` = Object.keys / values / entries.

Q17. What is a tuple?

DEFINITION

An ordered collection that cannot be changed after creation (immutable). Written with ( ).

Used for fixed groups of values and for returning several values from a function.

PYTHON EXAMPLE

```
point = (10, 20)
x, y = point             # unpacking (like destructuring)

def build_sql(filters):
    return "SELECT ...", ["@dept"]   # returns a tuple

sql, params = build_sql(filters)
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
const [x, y] = [10, 20] as const;
```

Q18. What is a set?

DEFINITION

An unordered collection of unique values. Duplicates are removed automatically.

PYTHON EXAMPLE

```
departments = {"hr", "it", "hr"}     # {'hr', 'it'}
departments.add("finance")
"it" in departments                  # very fast lookup
unique_ids = set(["a", "b", "a"])    # {'a', 'b'}
{1, 2} | {2, 3}    # union        {1, 2, 3}
{1, 2} & {2, 3}    # intersection {2}
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
const departments = new Set(['hr', 'it', 'hr']);
```

Tip: Careful: `{}` is an empty dict. An empty set is `set()`.

Q19. What is unpacking (destructuring) and the * / ** spread?

DEFINITION

Taking values out of a list/tuple/dict into variables, or spreading them into a new one.

`*` spreads lists, `**` spreads dicts - just like `...` in TypeScript.

PYTHON EXAMPLE

```
first, *rest = [1, 2, 3, 4]        # first=1, rest=[2, 3, 4]
messages = [*history, new_message]   # [...history, newMessage]
item = {**metadata, "score": 0.9}   # {...metadata, score: 0.9}
a, b = b, a                        # swap two variables
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
const [first, ...rest] = [1, 2, 3, 4];
const messages = [...history, newMessage];
const item = { ...metadata, score: 0.9 };
```

Q20. What are list and dict comprehensions?

DEFINITION

A short way to build a new list/dict from another collection, with an optional filter. Python's version of .map() and .filter().

Read it as: '[what to keep for each item if condition]'. Comprehensions are everywhere in Python code, so learn them early.

PYTHON EXAMPLE

```
numbers = [1, 2, 3, 4, 5]
squares = [n * n for n in numbers]              # map
evens = [n for n in numbers if n % 2 == 0]      # filter
texts = [d.page_content for d in docs if d.metadata["score"] > 0.5]

by_id = {doc["id"]: doc for doc in items}       # dict comprehension
unique = {d["department"] for d in items}       # set comprehension
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
const squares = numbers.map(n => n * n);
const evens = numbers.filter(n => n % 2 === 0);
const byId = Object.fromEntries(items.map(d => [d.id, d]));
```

4. Control Flow

if, loops and pattern matching.

Q21. How do if / elif / else and the ternary work?

DEFINITION

`elif` = 'else if'. The one-line ternary is written `a if condition else b`.

PYTHON EXAMPLE

```
if score >= 0.8:
    label = "strong"
elif score >= 0.5:
    label = "ok"
else:
    label = "weak"

mode = "agent" if use_agent else "pipeline"     # ternary
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
const mode = useAgent ? 'agent' : 'pipeline';
```

Q22. How do for loops work? What are range, enumerate and zip?

DEFINITION

`for x in collection` loops over values directly (like for...of). Helpers give you numbers, indexes and pairs.

- `range(n)` -> 0..n-1 (classic counter loop)
- `enumerate(list)` -> (index, value) pairs
- `zip(a, b)` -> pairs items from two lists side by side

PYTHON EXAMPLE

```
for doc in docs:
    print(doc)

for i in range(3):                      # 0, 1, 2
    print(i)

for n, doc in enumerate(docs, start=1):  # 1, 2, 3...
    print(f"[{n}] {doc}")

for chunk, vector in zip(chunks, vectors):
    save(chunk, vector)
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
for (const doc of docs) { ... }
docs.forEach((doc, i) => ...);
```

Q23. while, break and continue?

DEFINITION

`while` repeats while a condition is true. `break` exits a loop, `continue` skips to the next round.

PYTHON EXAMPLE

```
attempts = 0
while attempts < 3:
    attempts += 1          # no ++ in Python!
    if call_api():
        break

for msg in messages:
    if not msg.strip():
        continue           # skip empty messages
    handle(msg)
```

Tip: Python has no `i++`. Use `i += 1`.

Q24. What is match / case?

DEFINITION

Python 3.10+ pattern matching - a more powerful switch statement.

PYTHON EXAMPLE

```
match event["type"]:
    case "token":
        show(event["text"])
    case "error" | "blocked":
        show_error(event["message"])
    case _:                    # default
        pass                   # 'do nothing'
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
switch (event.type) {
  case 'token': ...; break;
  default: break;
}
```

Tip: `pass` is a placeholder that does nothing - needed because a block can't be empty.

5. Functions

Defining functions, arguments, lambdas and type hints.

Q25. How do I define a function?

DEFINITION

With `def name(parameters):`. Use `return` to send a value back (without it the function returns None).

PYTHON EXAMPLE

```
def add(a, b):
    return a + b

def chunk_count(text: str, size: int = 1800) -> int:   # default value + type hints
    return len(text) // size + 1

chunk_count("long text...")
chunk_count("long text...", size=500)    # keyword argument
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
function chunkCount(text: string, size = 1800): number {
  return Math.floor(text.length / size) + 1;
}
```

Tip: Keyword arguments (`size=500`) make calls readable - AI libraries use them everywhere: `ChatOpenAI(model=..., temperature=0)`.

Q26. What are *args and **kwargs?

DEFINITION

`*args` collects extra positional arguments into a tuple; `**kwargs` collects extra named arguments into a dict.

Used when a function accepts 'any number of arguments' or passes them on to another function.

PYTHON EXAMPLE

```
def log(*args, **kwargs):
    print(args)      # (1, 2)
    print(kwargs)    # {'level': 'info'}

log(1, 2, level="info")

settings = {"model": "gpt-4o-mini", "temperature": 0}
llm = ChatOpenAI(**settings)    # spread a dict as named arguments
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
function log(...args: unknown[]) { }
```

Q27. What are lambda functions?

DEFINITION

Small one-line anonymous functions: `lambda params: expression`. Like a short arrow function.

Mostly used as the `key` for sorting.

PYTHON EXAMPLE

```
square = lambda x: x * x

docs.sort(key=lambda d: d["score"], reverse=True)    # highest score first
sorted(files, key=lambda f: (f["name"], f["page"]))
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
const square = (x: number) => x * x;
docs.sort((a, b) => b.score - a.score);
```

Tip: A lambda can only hold one expression. For anything longer, write a normal `def`.

Q28. What are type hints?

DEFINITION

Optional type labels on variables and functions. Python does NOT check them at runtime - editors and tools like mypy do.

Coming from TypeScript you'll love them. In AI code they matter even more: LangChain and Pydantic read them to build schemas that the LLM fills in.

PYTHON EXAMPLE

```
from typing import Literal

def search(query: str, top_k: int = 5) -> list[dict]:
    ...

year: int | None = None                  # number | null
mode: Literal["pipeline", "agent"]       # 'pipeline' | 'agent'
scores: dict[str, float] = {}            # Record<string, number>
vectors: list[list[float]]               # number[][]
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
function search(query: string, topK = 5): Record<string, unknown>[] { }
let year: number | null = null;
type Mode = 'pipeline' | 'agent';
```

Q29. What is a docstring?

DEFINITION

A string right under a `def` or `class` that describes it. Like JSDoc.

In AI agents, the docstring of a tool function is literally what the LLM reads to decide when to call it - so write it clearly!

PYTHON EXAMPLE

```
def semantic_search(query: str) -> str:
    """Search company documents by meaning.
    Use for questions about policies, e.g. 'leave days for interns'."""
    ...
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
/** Search company documents by meaning. */
function semanticSearch(query: string): string { }
```

Q30. What is variable scope? What are global and nonlocal?

DEFINITION

Scope = where a variable can be seen. Variables created inside a function exist only inside it.

A function can READ outer variables, but to REASSIGN them you must declare `global` or `nonlocal`. Changing a list/dict (append, set key) does not need it. In practice: avoid `global`.

PYTHON EXAMPLE

```
count = 0
def increment():
    global count       # needed because we reassign count
    count += 1

found = {}
def remember(doc):
    found[doc["id"]] = doc   # mutating - no keyword needed
```

Q31. What is a closure?

DEFINITION

An inner function that remembers variables from the function it was created in.

Exactly like a JavaScript arrow function capturing an outer variable. Used in AI agents to give tool functions access to per-request data.

PYTHON EXAMPLE

```
def make_multiplier(factor):
    def multiply(x):
        return x * factor     # remembers factor
    return multiply

double = make_multiplier(2)
double(5)    # 10
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
const makeMultiplier = (factor: number) => (x: number) => x * factor;
```

6. Classes & Objects (OOP)

Python classes work like TypeScript classes, with a few different words.

Q32. How do I write a class?

DEFINITION

`class Name:` with an `__init__` method as the constructor. `self` is the same as `this`.

Every method takes `self` as its first parameter - Python passes it automatically. Attributes are created by assigning to `self.something` inside `__init__`.

PYTHON EXAMPLE

```
class ChatSession:
    def __init__(self, user: str):       # constructor
        self.user = user
        self.history: list[dict] = []

    def add(self, role: str, content: str) -> None:
        self.history.append({"role": role, "content": content})

session = ChatSession("priya")           # no 'new' keyword
session.add("user", "Hi")
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
class ChatSession {
  history: Message[] = [];
  constructor(public user: string) {}
  add(role: string, content: string) { this.history.push({ role, content }); }
}
const session = new ChatSession('priya');
```

Q33. Is there public / private?

DEFINITION

No access modifiers. By convention, a name starting with _ means 'private, don't use from outside'.

Python trusts developers ('we are all adults here'). `_load()` in a module or `self._cache` in a class signals 'internal'.

PYTHON EXAMPLE

```
class Store:
    def __init__(self):
        self._items = []        # 'private' by convention

    def _load(self):            # internal helper
        ...
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
class Store { private items: Item[] = []; private load() {} }
```

Q34. How does inheritance work?

DEFINITION

`class Child(Parent):` - the child gets all parent methods. `super()` calls the parent.

LangChain uses this a lot: you write a custom retriever by inheriting `BaseRetriever` and implementing one method.

PYTHON EXAMPLE

```
class BaseRetriever:
    def invoke(self, query):
        return self._get_relevant_documents(query)

class MyRetriever(BaseRetriever):
    def _get_relevant_documents(self, query):
        return ["doc about " + query]

MyRetriever().invoke("leave")    # ['doc about leave']
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
class MyRetriever extends BaseRetriever { ... }
```

Q35. What are 'dunder' (magic) methods like __str__ and __len__?

DEFINITION

Special methods with double underscores that Python calls automatically for built-in operations.

- `__init__` -> constructor
- `__str__` -> what `print(obj)` shows (like toString)
- `__repr__` -> developer view in the debugger
- `__len__` -> `len(obj)`, `__eq__` -> `==`, `__or__` -> the `|` operator

LangChain's `prompt | model | parser` works because of `__or__`.

PYTHON EXAMPLE

```
class Chunk:
    def __init__(self, text):
        self.text = text
    def __str__(self):
        return f"Chunk({self.text[:20]}...)"
    def __len__(self):
        return len(self.text)

print(Chunk("Employees get 24 days"))   # Chunk(Employees get 24 da...)
```

Q36. What are @property, @staticmethod and @classmethod?

DEFINITION

@property = a getter you read like a field. @staticmethod = no self, just a function in the class. @classmethod = gets the class (`cls`) instead of the instance - often used as a factory.

PYTHON EXAMPLE

```
class User:
    def __init__(self, first, last):
        self.first, self.last = first, last

    @property
    def full_name(self):          # user.full_name  (no brackets)
        return f"{self.first} {self.last}"

    @staticmethod
    def is_valid_email(email):
        return "@" in email

    @classmethod
    def from_dict(cls, data):     # User.from_dict({...})
        return cls(data["first"], data["last"])
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
get fullName() { return `${this.first} ${this.last}`; }
static isValidEmail(email: string) { ... }
```

Q37. What is a dataclass?

DEFINITION

A decorator that writes `__init__`, `__repr__` and `__eq__` for you from typed fields.

Perfect for simple data holders, like a TypeScript interface that is also a class.

PYTHON EXAMPLE

```
from dataclasses import dataclass, field

@dataclass
class Source:
    file_name: str
    page: int
    score: float | None = None
    tags: list[str] = field(default_factory=list)   # safe default for lists

s = Source("leave.pdf", 3)
print(s)    # Source(file_name='leave.pdf', page=3, score=None, tags=[])
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
interface Source { fileName: string; page: number; score?: number; }
```

Tip: For data coming from users, APIs or LLMs, use **Pydantic** instead (it also validates) - see Section 11.

Q38. Abstract classes and Protocols - how do I write an 'interface'?

DEFINITION

ABC = a base class with abstract methods that children must implement. Protocol = structural typing, like a TS interface (any class with the right methods fits).

PYTHON EXAMPLE

```
from abc import ABC, abstractmethod
from typing import Protocol

class VectorStore(ABC):
    @abstractmethod
    def search(self, vector: list[float], k: int) -> list[dict]: ...

class Embedder(Protocol):              # 'duck typing' interface
    def embed(self, text: str) -> list[float]: ...
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
interface Embedder { embed(text: string): number[]; }
abstract class VectorStore { abstract search(v: number[], k: number): Doc[]; }
```

7. Modules, Packages & Imports

How Python code is split into files and folders.

Q39. How do imports work?

DEFINITION

Every .py file is a module. You import a whole module or specific names from it.

There is no `export` keyword - every top-level name in a file can be imported.

PYTHON EXAMPLE

```
import json                                  # whole module
json.dumps({"a": 1})

from datetime import datetime, timezone       # specific names
from backend.chat.merge import merge_results   # your own file
import numpy as np                           # alias
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
import * as json from './json';
import { mergeResults } from './chat/merge';
```

Q40. What is a package and __init__.py?

DEFINITION

A package is a folder of modules. An `__init__.py` file marks it as a package and runs on first import (like an index.ts barrel file).

PYTHON EXAMPLE

```
backend/
    __init__.py
    config.py
    chat/
        __init__.py
        pipeline.py

from backend.chat.pipeline import run_pipeline
```

Q41. What does if __name__ == "__main__": mean?

DEFINITION

Code inside this block runs only when you run the file directly, not when another file imports it.

It lets one file be both a reusable module and a runnable script.

PYTHON EXAMPLE

```
def ingest_file(path):
    ...

if __name__ == "__main__":      # python ingest.py report.pdf
    import sys
    ingest_file(sys.argv[1])
```

Q42. Which standard library modules should I know?

DEFINITION

Python ships with 'batteries included' - many useful modules with no install.

- `os`, `pathlib` - files, folders, environment variables
- `json` - parse/create JSON
- `datetime` - dates and times
- `re` - regular expressions
- `logging` - proper logs instead of print
- `functools` - `lru_cache`, `partial`
- `collections` - `Counter`, `defaultdict`
- `asyncio` - async programming
- `typing` - type hints
- `argparse` - command-line arguments

PYTHON EXAMPLE

```
from collections import Counter
Counter(["hr", "it", "hr"])        # Counter({'hr': 2, 'it': 1})

import re
re.search(r"\d{4}", "Policy 2025").group()   # '2025'
```

8. Error Handling

try/except is Python's try/catch.

Q43. How do try / except / finally work?

DEFINITION

`try` runs risky code; `except` catches errors (like catch); `finally` always runs; `else` runs only when there was no error.

Catch **specific** error types first. Never silently swallow every error.

PYTHON EXAMPLE

```
try:
    data = json.loads(text)
except json.JSONDecodeError as error:
    print("Bad JSON:", error)
    data = {}
except Exception:                 # anything else
    logger.exception("Unexpected")
    raise                          # re-throw the same error
finally:
    print("done")
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
try { data = JSON.parse(text); }
catch (error) { ... }
finally { ... }
```

Q44. How do I raise my own errors?

DEFINITION

`raise SomeError("message")` = throw. Custom errors are classes that inherit from Exception.

PYTHON EXAMPLE

```
class GuardrailBlocked(Exception):
    pass

def check(question: str):
    if len(question) > 2000:
        raise ValueError("Question too long")
    if "ignore previous instructions" in question.lower():
        raise GuardrailBlocked("prompt injection")
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
throw new Error('Question too long');
class GuardrailBlocked extends Error {}
```

Q45. What is 'with' (a context manager)?

DEFINITION

A block that sets up a resource and ALWAYS cleans it up afterwards - even if an error happens.

Most common use: opening files. The file is closed automatically at the end of the block.

PYTHON EXAMPLE

```
with open("notes.txt", encoding="utf-8") as file:
    text = file.read()
# file is closed here automatically
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
// similar to try { ... } finally { file.close(); }
```

9. Files, JSON & Environment Variables

Reading documents, saving data and handling secrets - daily work in AI apps.

Q46. How do I read and write text files?

DEFINITION

`open(path, mode)` inside a `with` block. Modes: "r" read, "w" write (replace), "a" append, "rb" read bytes.

PYTHON EXAMPLE

```
with open("policy.md", encoding="utf-8") as f:
    text = f.read()              # whole file as one string

with open("policy.md", encoding="utf-8") as f:
    for line in f:               # line by line (good for big files)
        print(line.strip())

with open("out.txt", "w", encoding="utf-8") as f:
    f.write("Hello")
```

Tip: Always pass `encoding="utf-8"` on Windows, or special characters can break.

Q47. How do I work with JSON?

DEFINITION

The `json` module converts between JSON text and Python dicts/lists.

- `json.loads(text)` -> dict (like JSON.parse)
- `json.dumps(obj)` -> text (like JSON.stringify)
- `json.load(file)` / `json.dump(obj, file)` -> read/write files

PYTHON EXAMPLE

```
import json

data = json.loads('{"name": "leave.pdf", "pages": 3}')
data["pages"]                        # 3
text = json.dumps(data, indent=2)    # pretty JSON string

with open("store.json", "w", encoding="utf-8") as f:
    json.dump(items, f)
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
const data = JSON.parse(text);
JSON.stringify(data, null, 2);
```

Tip: JSON true/false/null become Python True/False/None automatically.

Q48. What is pathlib?

DEFINITION

A modern, object-oriented way to work with file paths that works on Windows, Mac and Linux.

PYTHON EXAMPLE

```
from pathlib import Path

docs = Path("sample-docs")
for file in docs.glob("*.md"):        # all markdown files
    print(file.name, file.suffix)    # 'hr.md', '.md'
    text = file.read_text(encoding="utf-8")

Path("data").mkdir(exist_ok=True)    # create folder if missing
path = Path("data") / "store.json"    # join paths with /
```

Q49. How do I store API keys safely (environment variables, .env)?

DEFINITION

Secrets live in environment variables, never in code. A `.env` file holds them locally and is never committed to git.

It is the same idea as Angular's `environment.ts`, but secrets come from the machine, not the code.

PYTHON EXAMPLE

```
# .env  (add .env to .gitignore!)
AZURE_OPENAI_API_KEY=abc123
TOP_K=5

# config.py
import os
from dotenv import load_dotenv     # pip install python-dotenv

load_dotenv()
API_KEY = os.getenv("AZURE_OPENAI_API_KEY", "")
TOP_K = int(os.getenv("TOP_K", "5"))    # env values are always strings
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
// environment.ts
export const environment = { apiUrl: '...' };
```

10. Must-Know Advanced Python

These features appear in almost every AI codebase. Take your time here.

Q50. What is an iterator / iterable?

DEFINITION

An iterable is anything you can loop over (list, dict, string, file). An iterator gives the values one at a time with `next()`.

PYTHON EXAMPLE

```
numbers = iter([1, 2, 3])
next(numbers)    # 1
next(numbers)    # 2
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
const it = [1, 2, 3][Symbol.iterator]();
it.next();
```

Q51. What is a generator and yield?

DEFINITION

A function that uses `yield` to give back values one by one, pausing between them, instead of returning a full list at once.

Two big benefits: it uses little memory, and the caller gets each value **immediately**. That is how chatbots stream an answer word by word. Think of it as a simple, pull-based Observable.

PYTHON EXAMPLE

```
def stream_answer():
    yield "Employees "
    yield "get 24 "
    yield "days."

for piece in stream_answer():
    print(piece, end="")    # prints piece by piece

squares = (n * n for n in range(1_000_000))   # generator expression: lazy
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
function* streamAnswer() {
  yield 'Employees ';
  yield 'get 24 days.';
}
```

Tip: `yield from other_generator()` passes on all values of another generator.

Q52. What is a decorator?

DEFINITION

A function that wraps another function to add behaviour. You apply it with `@name` above a def.

Very similar to Angular decorators like `@Component` or `@Injectable` - they add behaviour without changing the function's code. AI libraries use them a lot: `@tool`, `@app.get`, `@lru_cache`.

PYTHON EXAMPLE

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time() - start:.2f}s")
        return result
    return wrapper

@timer                    # same as: ask_llm = timer(ask_llm)
def ask_llm(question):
    ...
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
@Injectable({ providedIn: 'root' })
export class ChatService {}
```

Q53. What is @lru_cache?

DEFINITION

A built-in decorator that remembers a function's result for each set of arguments, so the next call returns instantly.

AI apps use it to create expensive clients (LLM, database) only once - like a singleton service with `providedIn: 'root'`.

PYTHON EXAMPLE

```
from functools import lru_cache

@lru_cache
def get_chat_model():
    print("creating client...")       # printed only once
    return AzureChatOpenAI(...)

get_chat_model()   # creates
get_chat_model()   # cached - same object
```

Q54. What is async / await in Python?

DEFINITION

A way to run many slow I/O tasks (API calls, DB queries) at the same time on one thread, without blocking. Same idea as Promises + async/await in TypeScript.

- `async def` creates a coroutine (like an async function returning a Promise).
- `await` waits for it.
- `asyncio.gather()` runs several at once (like `Promise.all`).
- `asyncio.run()` starts the event loop in a script (Node does this for you).

PYTHON EXAMPLE

```
import asyncio

async def ask(question: str) -> str:
    response = await llm.ainvoke(question)   # 'a' = async version
    return response.content

async def main():
    answers = await asyncio.gather(          # in parallel
        ask("What is RAG?"), ask("What is an agent?"))
    print(answers)

asyncio.run(main())
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
const answers = await Promise.all([ask('What is RAG?'), ask('What is an agent?')]);
```

Tip: Never call slow normal (blocking) code inside `async def` - it freezes everything. Use async libraries or run it in a thread.

Q55. Threads, processes or async - what is the GIL?

DEFINITION

The GIL (Global Interpreter Lock) lets only one thread run Python code at a time.

- Waiting on networks/APIs/databases (most AI integration work) -> **async** or threads are fine.
- Heavy CPU maths -> **multiprocessing**, or libraries like NumPy that do the work in C.

PYTHON EXAMPLE

```
from concurrent.futures import ThreadPoolExecutor

with ThreadPoolExecutor(max_workers=5) as pool:
    results = list(pool.map(embed_text, texts))   # 5 API calls at a time
```

Q56. How do I log properly instead of using print?

DEFINITION

The `logging` module writes messages with a level (DEBUG, INFO, WARNING, ERROR) and timestamps.

Logs can be sent to files or Azure Application Insights, and you can turn levels on/off without changing code.

PYTHON EXAMPLE

```
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("chatbot")

logger.info("Question received")
logger.warning("Slow response: %.1fs", 8.2)
try:
    ...
except Exception:
    logger.exception("Answering failed")   # includes the full stack trace
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
console.info(...); console.warn(...); console.error(...);
```

11. Key Libraries for AI Developers

The packages you will install in almost every AI project.

Q57. What is Pydantic and why is it everywhere in AI code?

DEFINITION

A library to define data models with type hints that are VALIDATED at runtime. Like a TypeScript interface that also checks the data.

Three everyday uses:

- Validate API request bodies (FastAPI uses it automatically).
- Validate config and data from files.
- Tell an LLM exactly which JSON shape to return (structured output).

PYTHON EXAMPLE

```
from pydantic import BaseModel, Field, ValidationError

class Question(BaseModel):
    text: str
    mode: str = "pipeline"
    top_k: int = Field(default=5, ge=1, le=20)   # between 1 and 20

q = Question(text="Leave days?", top_k="3")    # "3" converted to int 3
q.model_dump()        # {'text': 'Leave days?', 'mode': 'pipeline', 'top_k': 3}

try:
    Question(top_k=99)
except ValidationError as e:
    print(e)          # text: missing, top_k: must be <= 20
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
interface Question { text: string; mode?: string; topK?: number; }
// TS interfaces disappear at runtime - no validation!
```

Q58. How do I call a REST API (requests / httpx)?

DEFINITION

`requests` is the classic HTTP client (like HttpClient/fetch). `httpx` is the same but also supports async.

PYTHON EXAMPLE

```
import httpx

response = httpx.get("https://api.example.com/users",
                     params={"page": 1},
                     headers={"Authorization": f"Bearer {token}"},
                     timeout=30)
response.raise_for_status()     # throws on 4xx/5xx
users = response.json()         # -> list/dict

httpx.post(url, json={"question": "Hi"})
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
this.http.get<User[]>(url, { params: { page: 1 } }).subscribe(...);
```

Tip: Always set a timeout for external calls - LLM APIs can be slow.

Q59. What is FastAPI?

DEFINITION

A modern Python web framework to build APIs. Think 'NestJS for Python'.

You write normal functions; type hints + Pydantic give you validation and automatic Swagger docs at `/docs`. It's the most common way to put an AI pipeline behind an API that your Angular app calls.

PYTHON EXAMPLE

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class AskRequest(BaseModel):
    question: str

@app.get("/api/health")
def health():
    return {"status": "ok"}              # dict -> JSON

@app.post("/api/ask")
async def ask(body: AskRequest):           # validated automatically
    answer = await chain.ainvoke({"question": body.question})
    return {"answer": answer}

# run:  uvicorn main:app --reload --port 8000
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
@Controller('api')
export class AskController {
  @Post('ask') ask(@Body() body: AskDto) { ... }
}
```

Tip: Enable CORS (`CORSMiddleware`) so your Angular dev server on port 4200 can call it.

Q60. What is NumPy?

DEFINITION

The library for fast maths on arrays of numbers. Embeddings (vectors) are usually NumPy arrays.

Operations work on the whole array at once (no loops), and run in fast C code.

PYTHON EXAMPLE

```
import numpy as np

a = np.array([0.1, 0.3, 0.5])
b = np.array([0.2, 0.1, 0.4])
a + b                 # array([0.3, 0.4, 0.9])  element by element
np.dot(a, b)          # dot product
np.linalg.norm(a)     # vector length

def cosine(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

Q61. What is pandas?

DEFINITION

A library for tables of data (DataFrames) - like an Excel sheet in code.

Used for loading CSV/Excel data, cleaning it, and analysing results (e.g. evaluation scores of your chatbot).

PYTHON EXAMPLE

```
import pandas as pd

df = pd.read_csv("eval_results.csv")   # columns: question, correct, latency
df.head()                              # first 5 rows
df["correct"].mean()                   # accuracy, e.g. 0.87
slow = df[df["latency"] > 5]           # filter rows
df.groupby("category")["latency"].mean()
```

12. Python for LLM & AI Apps

Now we put it together: real AI developer code. Each example is short on purpose.

Q62. How do I call an LLM from Python (OpenAI / Azure OpenAI SDK)?

DEFINITION

You send a list of messages (system + user) to the chat completions API and get the answer text back.

Roles: **system** = the rules, **user** = the question, **assistant** = earlier answers. The model has no memory, so you send the conversation every time.

PYTHON EXAMPLE

```
from openai import AzureOpenAI

client = AzureOpenAI(azure_endpoint=ENDPOINT, api_key=KEY, api_version="2024-10-21")

response = client.chat.completions.create(
    model="gpt-4o-mini",               # Azure: your deployment name
    temperature=0.2,
    messages=[
        {"role": "system", "content": "You are a helpful HR assistant."},
        {"role": "user", "content": "How many leave days do I get?"},
    ],
)
print(response.choices[0].message.content)
print(response.usage.total_tokens)    # tokens = cost
```

Tip: Temperature 0 = same answer every time (good for classification); higher = more creative.

Q63. How do I stream the answer word by word?

DEFINITION

Add `stream=True`; you get a generator of small pieces (chunks) instead of one big response.

Users see the first words after ~1 second instead of waiting for the full answer. Your backend forwards each piece to Angular (WebSocket or Server-Sent Events).

PYTHON EXAMPLE

```
stream = client.chat.completions.create(model="gpt-4o-mini", messages=messages,
                                        stream=True)
for chunk in stream:
    if chunk.choices and chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="", flush=True)
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
// Angular side: append each token to the message text
this.socket.events$.subscribe(e => { if (e.type === 'token') msg.text += e.text; });
```

Q64. How do I create embeddings and compare them?

DEFINITION

An embedding turns text into a list of numbers that represents its meaning. Similar meaning -> similar numbers (high cosine similarity).

This is the core of semantic search: embed your documents once, embed each question, and find the closest documents.

PYTHON EXAMPLE

```
import numpy as np

def embed(text: str) -> np.ndarray:
    result = client.embeddings.create(model="text-embedding-3-small", input=text)
    return np.array(result.data[0].embedding)       # 1536 numbers

q = embed("How many holidays do I get?")
d = embed("Employees receive 24 days of paid leave")
score = np.dot(q, d) / (np.linalg.norm(q) * np.linalg.norm(d))
print(round(score, 2))    # high, even with no shared words
```

Q65. What does a basic LangChain chain look like?

DEFINITION

LangChain connects a prompt template, a model and an output parser with the `|` pipe operator.

If you know RxJS `.pipe()`, this will feel natural: data flows left to right through each step. Every chain has `.invoke()` (one result) and `.stream()` (pieces).

PYTHON EXAMPLE

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import AzureChatOpenAI

prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer only from this context:\n{context}"),
    ("human", "{question}"),
])
llm = AzureChatOpenAI(azure_deployment="gpt-4o-mini", temperature=0)
chain = prompt | llm | StrOutputParser()

chain.invoke({"context": "Employees get 24 days.", "question": "Leave days?"})
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
source$.pipe(map(buildPrompt), switchMap(callModel), map(toText));
```

Q66. How do I get JSON / typed objects back from an LLM (structured output)?

DEFINITION

Give the model a Pydantic class; it must return data in that shape, and you get a real object back.

No more 'please answer in JSON' and fragile `json.loads()`. The field descriptions act as instructions.

PYTHON EXAMPLE

```
from typing import Literal
from pydantic import BaseModel, Field

class Ticket(BaseModel):
    category: Literal["hardware", "software", "access"]
    priority: Literal["low", "high"]
    summary: str = Field(description="One short sentence")

structured_llm = llm.with_structured_output(Ticket)
ticket = structured_llm.invoke("My laptop won't start and I have a demo in 1 hour!")
ticket.priority     # 'high'   <- a typed field, not text
```

Q67. What is a tool, and how does an agent use it?

DEFINITION

A tool is a normal Python function the LLM is allowed to ask for. An agent is an LLM in a loop that decides which tools to call until it can answer.

The LLM never runs code itself. It replies 'please call get_weather with city=Pune', LangChain runs your function, and sends the result back to the LLM. The docstring and type hints tell the model what the tool does.

PYTHON EXAMPLE

```
from langchain.agents import create_agent
from langchain_core.tools import tool

@tool
def get_leave_balance(employee_id: str) -> int:
    """Return the remaining paid leave days of an employee."""
    return hr_api.balance(employee_id)

agent = create_agent(model=llm, tools=[get_leave_balance],
                     system_prompt="You are an HR assistant.")
result = agent.invoke({"messages": [("user", "How many days left for E42?")]})
print(result["messages"][-1].content)
```

Q68. What does a minimal RAG flow look like in Python?

DEFINITION

RAG = Retrieve relevant text from your documents, Augment the prompt with it, Generate the answer.

Steps: split documents into chunks -> embed chunks once -> for each question embed it, find the closest chunks, and pass them to the LLM with the question.

PYTHON EXAMPLE

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 1. ingest (once)
splitter = RecursiveCharacterTextSplitter(chunk_size=1800, chunk_overlap=200)
chunks = splitter.split_text(open("hr-policy.md", encoding="utf-8").read())
vectors = [embed(c) for c in chunks]

# 2. retrieve (every question)
def retrieve(question: str, k: int = 3) -> list[str]:
    q = embed(question)
    scored = [(cosine(q, v), c) for c, v in zip(chunks, vectors)]
    return [c for _, c in sorted(scored, reverse=True)[:k]]

# 3. generate
context = "\n\n".join(retrieve("How many leave days?"))
chain.invoke({"context": context, "question": "How many leave days?"})
```

Tip: In production the vectors live in a vector database (Cosmos DB, Azure AI Search, pgvector) instead of a Python list.

Q69. How do I handle errors and retries when calling AI APIs?

DEFINITION

AI APIs fail sometimes: rate limits (429), timeouts, content filters. Retry with a growing wait (exponential backoff) and show users a friendly message.

The OpenAI SDK and LangChain already retry a few times (`max_retries`). For your own calls you can use the `tenacity` library.

PYTHON EXAMPLE

```
from openai import RateLimitError
from tenacity import retry, retry_if_exception_type, stop_after_attempt, wait_exponential

@retry(retry=retry_if_exception_type(RateLimitError),
       wait=wait_exponential(min=1, max=20),
       stop=stop_after_attempt(5))
def ask(messages):
    return client.chat.completions.create(model="gpt-4o-mini", messages=messages)
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
this.http.get(url).pipe(retry({ count: 3, delay: 1000 }));
```

13. Testing & Code Quality

How to keep Python code clean and safe to change.

Q70. How do I write tests with pytest?

DEFINITION

pytest finds files named `test_*.py` and runs functions named `test_*`. You check results with plain `assert`.

Much simpler than Jasmine/Karma - no describe/it boilerplate needed.

PYTHON EXAMPLE

```
# test_merge.py
from backend.chat.merge import merge_results

def test_empty_lists_give_empty_result():
    assert merge_results([], []) == []

def test_rejects_long_question():
    result = check_rules("x" * 5000)
    assert result.category == "too_long"

# run:  pytest -v
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
describe('merge', () => {
  it('returns empty', () => expect(mergeResults([], [])).toEqual([]));
});
```

Q71. How do I test code that calls an LLM?

DEFINITION

Replace (mock) the LLM with a fake that returns a fixed answer, so tests are fast, free and repeatable.

Test your own logic with mocks. Test the LLM's quality separately with an evaluation set of real questions and expected answers.

PYTHON EXAMPLE

```
from unittest.mock import patch

def test_pipeline_uses_router_result():
    with patch("backend.chat.pipeline.route_question") as fake_router:
        fake_router.return_value = RouteDecision(intent="semantic",
                                                 search_text="leave days")
        events = list(run_pipeline("leave?", []))
    assert any(e["type"] == "done" for e in events)
```

IN TYPESCRIPT (WHAT YOU ALREADY KNOW)

```
spyOn(service, 'route').and.returnValue(of(fakeDecision));
```

Q72. Which code quality tools should I use?

DEFINITION

Formatters, linters and type checkers - the Python versions of Prettier, ESLint and tsc.

- **ruff** - very fast linter + formatter (replaces flake8, isort, black)
- **mypy** or **pyright** - type checking, like `tsc --noEmit`
- **pre-commit** - run them before every commit (like husky)
- VS Code: install the Python + Pylance + Ruff extensions

PYTHON EXAMPLE

```
pip install ruff mypy
ruff check .        # lint      (eslint)
ruff format .       # format    (prettier)
mypy backend        # types     (tsc)
```

Q73. What does a typical AI project folder structure look like?

DEFINITION

Group code by responsibility, like feature modules in Angular.

PYTHON EXAMPLE

```
my-ai-app/
  .env                  # secrets (not in git)
  requirements.txt      # dependencies
  backend/
    main.py             # FastAPI app (routes)
    config.py           # settings from env vars
    models.py           # Pydantic models
    ingestion/          # load, split, embed, store documents
    chat/               # prompts, retriever, chains, agent
  tests/                # pytest tests
  web/                  # Angular frontend
```

14. TypeScript to Python Cheat Sheet

The same idea in both languages. Print this page and keep it next to you.

TypeScript / Angular	Python
`let x = 5; const Y = 5;`	`x = 5 Y = 5 (ALL_CAPS = constant)`
`null / undefined`	`None`
`true / false`	`True / False`
`&& || !`	`and or not`
`=== !==`	`== != (is None / is not None)`
`a ?? b`	`a if a is not None else b`
`cond ? a : b`	`a if cond else b`
`` `Hi ${name}` ``	`f"Hi {name}"`
`// comment`	`# comment`
`arr.length / str.length`	`len(arr) / len(s)`
`arr.push(x)`	`arr.append(x)`
`arr.map(x => x * 2)`	`[x * 2 for x in arr]`
`arr.filter(x => x > 2)`	`[x for x in arr if x > 2]`
`arr.includes(x)`	`x in arr`
`arr.slice(-3)`	`arr[-3:]`
`[...a, ...b]`	`[*a, *b]`
`{ ...obj, k: 1 }`	`{**obj, "k": 1}`
`const [a, b] = arr;`	`a, b = arr`
`obj.key / obj?.key ?? d`	`d["key"] / d.get("key", default)`
`Object.entries(obj)`	`d.items()`
`for (const x of arr)`	`for x in arr:`
`arr.forEach((x, i) => ...)`	`for i, x in enumerate(arr):`
`for (let i = 0; i < n; i++)`	`for i in range(n):`
`function f(a: number): string`	`def f(a: int) -> str:`
`(x) => x * 2`	`lambda x: x * 2`
`class A extends B`	`class A(B):`
`constructor() / this`	`def __init__(self): / self`
`new User()`	`User()`
`interface User { ... }`	`class User(BaseModel): ... (Pydantic)`
`'a' | 'b'`	`Literal["a", "b"]`
`number | null`	`int | None`
`try { } catch (e) { }`	`try: ... except Exception as e: ...`
`throw new Error('x')`	`raise ValueError("x")`
`async / await`	`async def / await`
`Promise.all([...])`	`await asyncio.gather(...)`
`JSON.parse / JSON.stringify`	`json.loads / json.dumps`
`console.log(x)`	`print(x) (or logging)`
`import { x } from './file';`	`from package.file import x`
`npm install / package.json`	`pip install / requirements.txt`
`node_modules`	`.venv (virtual environment)`

TypeScript / Angular	Python
`environment.ts`	`.env + os.getenv()`
`@Injectable providedIn root`	`@lru_cache function (singleton)`
`RxJS source$.pipe(a, b)`	`LangChain prompt | model | parser`
`HttpClient / fetch`	`httpx / requests`
`NestJS controller`	`FastAPI route (@app.get / @app.post)`
`Jasmine / Jest it(...)`	`pytest def test_...(): assert ...`
`ESLint / Prettier / tsc`	`ruff check / ruff format / mypy`

15. Your 6-Week Learning Roadmap

About 1-2 hours a day. Your Angular experience lets you move faster than a beginner - you're learning a new syntax, not how to program.

Week 1	**Python basics**	Sections 1-5. Variables, collections, loops, functions, comprehensions. Do 20-30 small exercises (e.g. on exercism.org or HackerRank).
Week 2	**Real Python**	Sections 6-10. Classes, modules, errors, files/JSON, generators, decorators, async. Write a script that reads all .md files in a folder and prints word counts.
Week 3	**APIs + data**	Section 11. Pydantic, httpx, FastAPI, NumPy. Build a FastAPI endpoint and call it from a small Angular page.
Week 4	**First LLM app**	Section 12. Call an LLM, stream the answer, structured output, embeddings and cosine similarity.
Week 5	**LangChain + RAG**	Chains, text splitters, retrievers, a vector store. Rebuild the genai-poc ingestion and pipeline from memory.
Week 6	**Agents + production**	Tools, create_agent, LangGraph basics, guardrails, pytest, logging, deploying to Azure.

Learn by doing: after each section, change one file in your genai-poc project and run it.

Read real code: your genai-poc backend has a comment on almost every line explaining the Python.

Use your strengths: you already know APIs, async, typing, testing and UI. Full-stack AI developers who can build both the Angular UI and the Python AI backend are in high demand.

Don't try to learn everything: data science topics (pandas deep-dives, model training, PyTorch) are optional for an AI integration role.